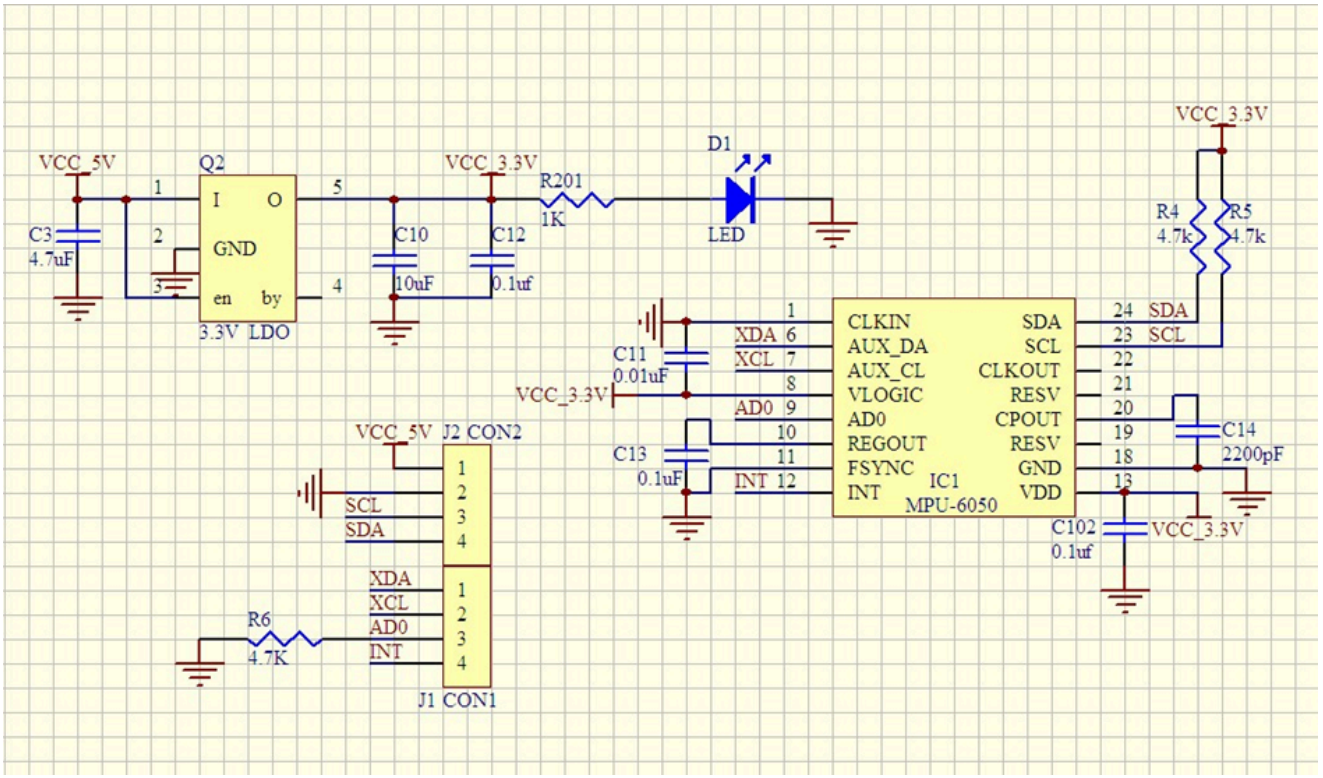
Q1

Include a screenshot of the **GY-521 module schematic** and explain why external pull-up resistors are not required in this setup. (You can find out the **module schematic** on the Internet)



解釋：

- 在GY-521模組中SDA以及SCL都有內建上拉電阻 `R4` 、 `R5` $4.7k\Omega$ ，因此外部不需要再使用上拉電阻。

Q2

- From the MPU6050 relevant documents, identify:
  - The register address used to read accelerometer data.

4.17 Registers 59 to 64 – Accelerometer Measurements
ACCEL_XOUT_H, ACCEL_XOUT_L, ACCEL_YOUT_H, ACCEL_YOUT_L, ACCEL_ZOUT_H, and ACCEL_ZOUT_L

Type: Read Only

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT[7:0]							

    - 儲存加速度值的暫存器位址從 `0x3B` 到 `0x40`
    - 每一個軸由兩個暫存器儲存資料
    - $(X_H, X_L, Y_H, Y_L, Z_H, Z_L)$一共6個Bytes。
  - The power management register address and specific bit used to wake up the device.

4.28 Register 107 – Power Management 1
PWR_MGMT_1

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
6B	107	DEVICE _RESET	SLEEP	CYCLE	-	TEMP_DIS	CLKSEL[2:0]		

    - 電源管理暫存器 (PWR_MGMT_1) 的位址為 `0x6B`。
    - 用來控制裝置睡眠模式的特定位元是 **Bit 6 (SLEEP)**，對此位元寫入0 即可解除睡眠模式並喚醒感測器。
  - The WHO_AM_I register address.

4.32 Register 117 – Who Am I
WHO_AM_I

Type: Read Only

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
75	117	-	WHO_AM_I[6:1]						-

    - 身份辨識暫存器 (WHO_AM_I) 的位址為 `0x75`。此暫存器會回傳設備位址
    - 預設回傳值為 `0x68`，如果 `AD0` 接至 V_{CC} 則會回傳 `0x69`

Q3

Explain the difference between START and repeated START conditions in terms of when they are used in the I2C communication sequence. Describe where each condition is used in your implementation.

- **START** 用在每次通訊的開頭，例如掃描位址或初始化寄存器。

```
void MPU6050_wakeup() //以 MPU6050_wakeup 為例
{

  // Your implemteation

  I2C_start();

  I2C_write_byte((MPU_ADDR << 1) | 0); // 傳送 SLA+W

  I2C_write_byte(PWR_MGMT_1);          // 指定 PWR_MGMT_1 暫存器 (0x6B)

  I2C_write_byte(0x00);                // 寫入 0x00 清除 Sleep bit

  I2C_stop();

}
```

- **Repeated START** 用在切換資料傳輸方向，不發送 STOP先將 SCL 與 SDA 釋放回 HIGH，再次發動一次 START 訊號。這樣匯流排不會被其他主機搶走。

```
uint8_t READ_WHO_AM_I_REG()//以READ_WHO_AM_I_REG()為例確認MPU6050設備位址
{
  // Your implementation
  I2C_start();
  I2C_write_byte((MPU_ADDR << 1) | 0); // 傳送 SLA+W
  I2C_write_byte(WHO_AM_I);            // 指定 WHO_AM_I 暫存器 (0x75)
  I2C_repeated_start();                // 重複啟動，準備改變資料方向
  I2C_write_byte((MPU_ADDR << 1) | 1); // 傳送 SLA+R
  // 讀取 1 byte，傳入 false 代表主機要回傳 NACK (結束讀取)
  uint8_t val = I2C_read_byte(false);
  I2C_stop();
  return val;
}
```

Q4

Explain the roles of ACK and NACK in the I2C protocol and how they are used in this assignment.

- ACK :
  - 接收端將 SDA 線拉至 LOW。Slave表示收到或是Master表示請繼續傳。

```
void address_scanner() //逐一掃瞄直到有設備發送ACK

{

  // Your implemteation

  int deviceCount = 0; // 用來記錄找到了幾個設備
  // 掃描所有 7-bit 位址

  for (uint8_t addr = 0; addr < 128; addr++) {

    I2C_start();
    uint8_t sla_w = (addr << 1); //LSB 是 R/W bit，寫入時為 0
    //發送位址並檢查是否有收到 ACK
    bool has_ack = I2C_write_byte(sla_w);
    I2C_stop();

    if (has_ack) {
      Serial.print("Addr: ");
      Serial.print(addr, DEC);      // 印出十進制
      Serial.print(" (0x");
      if (addr < 16) Serial.print("0");
      Serial.print(addr, HEX);      // 印出十六進制
      Serial.println(") ACK");
      deviceCount++;
    }
  }

  Serial.print("\nScan done. Found: ");
  Serial.print(deviceCount);
  Serial.println(" device(s)");

}
```

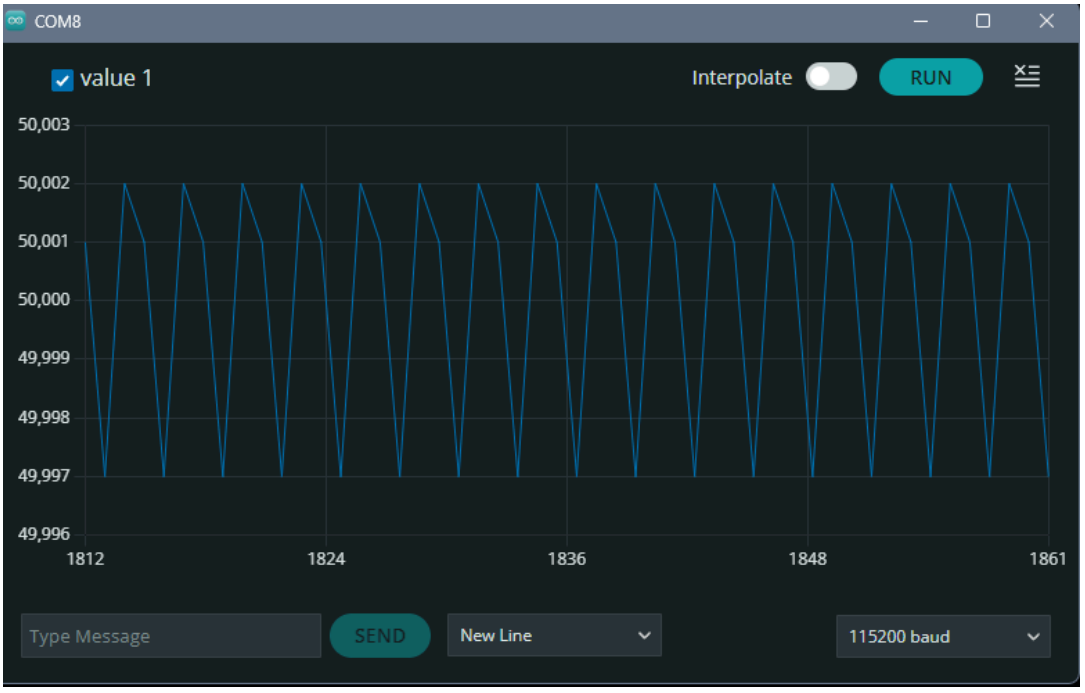
- NACK:
  - 接收端讓 SDA 線保持 HIGH。代表接收失敗。
  - 如果是接收端是Master表示讀完了。

```
bool readAccel_raw(int16_t *ax, int16_t *ay, int16_t *az)
{
  // Your implementation
  // Requirements:
  // 1. Read 6 consecutive bytes starting from ACCEL_XOUT_H.
  // 2. Use a repeated START to switch from write mode to read mode.
  // 3. Store the combined 16-bit signed results into *ax, *ay, and *az.
  // 4. Return true if the whole transaction succeeds; otherwise return
false.
  I2C_start();
  if (!I2C_write_byte((MPU_ADDR << 1) | 0))
    return false;
  if (!I2C_write_byte(ACCEL_XOUT_H))  //起始位置
    return false;
  I2C_repeated_start(); // 改變資料方向

  if (!I2C_write_byte((MPU_ADDR << 1) | 1))
    return false;
  uint8_t xh = I2C_read_byte(true);
  uint8_t xl = I2C_read_byte(true);
  uint8_t yh = I2C_read_byte(true);
  uint8_t yl = I2C_read_byte(true);
  uint8_t zh = I2C_read_byte(true);
  uint8_t zl = I2C_read_byte(false);// 最後一個不用ack，讀夠了
  I2C_stop();
  // 合併
  *ax = (int16_t)((xh << 8) | xl);
  *ay = (int16_t)((yh << 8) | yl);
  *az = (int16_t)((zh << 8) | zl);
  return true;
}
```

Q5

Measure and report the ADC sampling timing. Measure the time interval between two consecutive ADC samples and estimate the sampling period and sampling frequency. In addition, analyze the sampling jitter by showing how much the sampling interval varies over multiple measurements.



透過 `Micros` 函數 量測:

- Sampling period : $49997到50002\mu s$
- Sampling frequency: 約為$20HZ$
- jitter : 約 $\pm 3\mu s$

Q6

Measure the I2C transaction time. Measure the time required to complete one full I2C transaction for reading the WHO_AM_I register. The measurement should start at the generation of the START condition and end at the generation of the STOP condition. The measured time therefore represents the total duration of one complete transac tion from START to STOP.

```
Read five times of WHO_AM_I register.
reply = 0x68
WHO_AM_I Transaction Time (us): 1734
reply = 0x68
WHO_AM_I Transaction Time (us): 1735
reply = 0x68
WHO_AM_I Transaction Time (us): 1737
reply = 0x68
WHO_AM_I Transaction Time (us): 1735
reply = 0x68
WHO_AM_I Transaction Time (us): 1734

Success!
Got 5 replies of MPU address.
```

透過 `micros` 函數量測從START 到 STOP 約 1734至$1737\mu s$

Q7

Other interesting works or observations.

在實驗過程中，我觀察到當 MPU6050 處於完全靜止且水平放置時，三軸加速度的讀值並非完美的 $0, 0, 1g$ 會有些許誤差，導致LED會一直處於微亮的狀態。
以及在 Part 3 的 FSM 切換中 ，如果沒有處理好按鈕彈跳，模式切換會不穩定。

在實作I2C中了解到，如果有多個硬體設備位址相同可以透過 `AD0` 去改變位置，這揭示了 I2C 協定**匯流排位址空間受限且易衝突**。當系統需要整合多個相同功能的感測器時，固定的位址會導致無法在同一組匯流排上併行運作。